Politecnico di Milano

Compressed Sensing using Generative Models

Matteo Forlivesi
Renato Gallicola

1 Summary

1.1 Introduction

This experiment aims to evaluate the performance of compressed sensing using two models on the MNIST dataset: a Variational Auto-encoder (VAE) and a Generative Adversarial Network (GAN). The focus is on image reconstruction from compressed measurements. The MNIST dataset consists of 60,000 training images and 10,000 test images of handwritten digits, each of size 28x28 pixels.

1.2 Models and methodology

We employed two approaches to reconstruct the images from compressed measurements:

1. **Variational Auto-encoder (VAE):** This model is designed to generate images similar to those in the training set. The VAE compresses the input image into a lower-dimensional latent space and then reconstructs the image from this representation.
2. **Generative Adversarial Network (GAN):** This model is composed of two competing networks, a generative one and a discriminating one. They are trained to minimize their losses:
    - The generator tries to generate images from a random latent vector.
    - The discriminator tries to discern between real images and generated ones.

Each of the two was trained with a latent vector of dimension 20 and of dimension 30. We also trained the fully connected auto-encoder used by Bora et al., so that our results can be compared directly with the ones they publish.
For all the models, we used a random Gaussian matrix for the measurements. The compression levels varied from 10 to 750 measurements per image, out of the 784 pixels of a full image, allowing us to observe the reconstruction quality across different levels of compression.

1.3 Algorithm explanation

This section will explain the algorithm utilizing a generative model for compressed sensing.

Enhancements include a thousand gradient steps on the latent vector and 10 random restarts to improve convergence and error minimization.

1.4 The MNIST Dataset

The MNIST (Modified National Institute of Standards and Technology) dataset is a large collection of handwritten digits commonly used for training and evaluating image processing systems. It contains 60,000 training images and 10,000 test images, each being a gray scale image of size 28Œ28 pixels. Each image is labeled with a digit from 0 to 9, providing a diverse set of handwriting styles for developing robust digit recognition systems.

- **Size:** 60,000 training images and 10,000 test images.
- **Resolution:** Each image is 28Œ28 pixels, resulting in 784 features.
- **Classes:** 10 classes, representing digits 0 through 9.
- **Grayscale:** Pixel values range from 0 (black) to 255 (white).

The MNIST images were vectorized, resulting in 784-dimensional input vectors. Each pixel value was normalized to the range $[0, 1]$.

1.5 Results

The reconstruction performance was evaluated as the squared error between the original and the reconstructed image, divided by the number of pixels, averaged over ten test images, one for each digit class. The following observations were made:

1. **Performance with few measurements:** every generative prior is far more accurate than LASSO when measurements are scarce. The architecture of the reference paper reaches with 75 measurements the accuracy LASSO obtains with 400, a saving of a factor 5.3, within the range of 5 to 10 the authors report.
2. **Saturation:** the accuracy of the generative priors stops improving after roughly 200 measurements, because the reconstruction is confined to the range of the generator. From 500 measurements onwards LASSO becomes the most accurate method, as the reference paper predicts, and with 750 measurements it recovers the digits almost exactly.

3. **Comparison between the models:** the fully connected architecture of the paper is the most accurate of the priors and the cheapest of them to invert, while the DCGAN is the least accurate at almost every budget and about 73 times more expensive.

1.6 Conclusion

The experiment demonstrated the efficacy of using deep learning models, particularly VAEs, for image reconstruction from compressed measurements. These models significantly reduce the reconstruction error compared to traditional sparse recovery methods when the number of measurements is small, while the traditional method remains preferable when measurements are abundant. The findings highlight the potential of integrating compressed sensing techniques with deep learning to enhance image reconstruction quality in resource-constrained environments.
These results provide a promising direction for further research, particularly in optimizing measurement matrices and exploring other deep learning architectures for compressed sensing applications.

2 Introduction

Compressed sensing is a signal processing technique that enables the reconstruction of a signal from a small number of measurements. Given the measurement y, the measurements matrix $A \in \mathbb{R}^{m \times n}$ and the noise $\eta \in \mathbb{R}^m$, the goal is to estimate the vector $x^* \in \mathbb{R}^n$ such that:

$$y = Ax^* + \eta \tag{2.1}$$

Since $m \ll n$, this system is underdetermined, then it is not possible to find a unique solution. The NyquistShannon sampling theorem requires a certain amount of samples to perfectly reconstruct a signal, so compressed sensing exploits some features of the signal to address this problem in a context where the conditions of the aforementioned theorem are not met.

The standard version of the algorithm assumes the signal to be k-sparse, meaning that it has only k coefficients different from zero when represented in some domain. This assumption allows the signal retrieval through the use of algorithms for solving minimization problems, such as LASSO [11] for the L^1 norm minimization, under conditions on A established by the classical compressed sensing literature [2, 3].

Following Bora et al. [1], this experiment shifts from sparsity to structures derived from generative models, with the aim of assessing the goodness of these models when employed in compressed sensing. They feature a generator that maps vectors from a low dimensional to a high dimensional space, so from $z \in \mathbb{R}^k$ to $G(z) \in \mathbb{R}^n$. By training the generator, with high probability it will output vectors similar to the ones belonging to the dataset it has been trained on.

The idea behind the use of generative models in the context of compressed sensing is that we expect the vector x^* to be close to some $G(z)$, which can be found with an optimization algorithm based on gradient descent, that will be later discussed in detail.

3 Models

3.1 DCGAN

Generative Adversarial Networks (GANs) are a class of machine learning frameworks introduced by Goodfellow et al. in 2014 [4]. GANs have revolutionized the field of generative modeling, allowing for the creation of highly realistic synthetic data. This section will explore the fundamental concepts, structure, and applications of Deep Convolutional Generative Adversarial Networks (DCGANs).

3.1.1 Introduction to DCGANs

DCGANs consist of two neural networks: the generator and the discriminator, which compete to minimize their respective loss functions. The generator aims to create data that is indistinguishable from real data, while the discriminator attempts to differentiate between real and generated data. This adversarial process continues until the generator produces data that the discriminator cannot reliably distinguish from real data. DCGANs leverage convolutional neural networks (CNNs) to improve the quality of generated images.

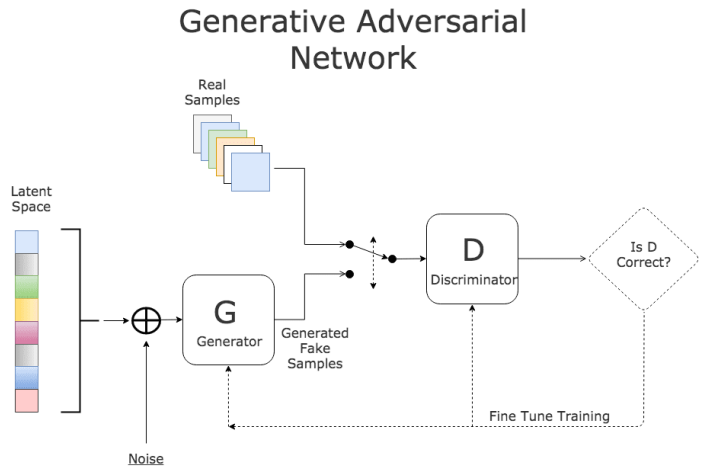

Figure 3.1: Structure of a Generative Adversarial Network, picture from [5]

3.1.2 Components of DCGANs

The two primary components of DCGANs are:

Generator

The generator, G, is a convolutional neural network that takes random noise, z, from a latent space and transforms it into a synthetic data sample, $G(z)$. The objective of the generator is to produce data that is as close as possible to the real data distribution. The generator's loss function is designed to minimize the likelihood of the discriminator correctly identifying the generated data as fake.

The actual structure of the generator, inspired by the standard DCGAN presented in [9], is reported in table 3.1.

Layer	**Output shape**	**Parameters**
Dense	1152	24,192
Reshape	$3 \times 3 \times 128$	–
Transposed convolution	$6 \times 6 \times 128$	262,272
LeakyReLU	$6 \times 6 \times 128$	–
Transposed convolution	$14 \times 14 \times 256$	524,544
LeakyReLU	$14 \times 14 \times 256$	–
Transposed convolution	$28 \times 28 \times 512$	2,097,664
LeakyReLU	$28 \times 28 \times 512$	–
Convolution	$28 \times 28 \times 1$	12,801
Total		**2,921,473**

Table 3.1: Structure of the DCGAN generator, shown for a latent dimension of 20. With a latent dimension of 30 only the first dense layer changes, and the total becomes 2,932,993 parameters.

Discriminator

The discriminator, D, is another convolutional neural network that evaluates data and assigns a probability that a given sample comes from the real data distribution rather than the generator. The discriminator's loss function aims to maximize the likelihood of correctly classifying real and fake data.

The actual structure of the discriminator, inspired by the standard DCGAN presented in [9], is reported in table 3.2.

Layer	Output shape	Parameters
Convolution	$14 \times 14 \times 64$	1,088
LeakyReLU	$14 \times 14 \times 64$	–
Convolution	$7 \times 7 \times 128$	131,200
LeakyReLU	$7 \times 7 \times 128$	–
Convolution	$4 \times 4 \times 128$	262,272
LeakyReLU	$4 \times 4 \times 128$	–
Flatten	2048	–
Dropout	2048	–
Dense	1	2,049
Total		**396,609**

Table 3.2: Structure of the DCGAN discriminator

3.1.3 Training Process

The training process for DCGANs involves optimizing the following value function, $V(G, D)$, which is a minimax game between G and D:

$$\min_G \max_D V(G, D) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))] \tag{3.1}$$

The training of DCGANs involves alternating between updating the discriminator and the generator. The discriminator is trained to maximize its classification accuracy, while the generator is trained to deceive the discriminator.

One detail of our implementation is worth mentioning: the labels given to the discriminator are perturbed by a small amount of uniform noise, drawn in $[0, 0.05]$. This keeps the discriminator from saturating early in training, which would leave the generator without a useful gradient.
The procedure can be summarized in the following steps:

1. Sample a batch of real data and a batch of random noise.
2. Update the discriminator by minimizing its classification loss on both real and generated data, which corresponds to maximizing $V(G, D)$.
3. Sample a new batch of random noise.
4. Update the generator by minimizing its loss function, which depends on the discriminator's performance.

3.1.4 Training Configuration

Both networks are trained on the full MNIST collection, obtained by concatenating the training and test splits into 70,000 images, since no labels are involved and the model is never evaluated on held out classes. Training runs for 50 epochs with a batch size of 32, using one Adam optimizer for each network with a learning rate of 0.0001 and a binary cross-entropy loss. As for the VAE, two versions were trained, one with a latent vector of dimension 20 and one of dimension 30.

3.2 VAE

3.2.1 Introduction to VAE

The **Variational Auto-encoder** (VAE) [7] is an artificial neural network whose encoder and decoder are connected through a probabilistic latent space, corresponding to the parameters of a variational distribution.

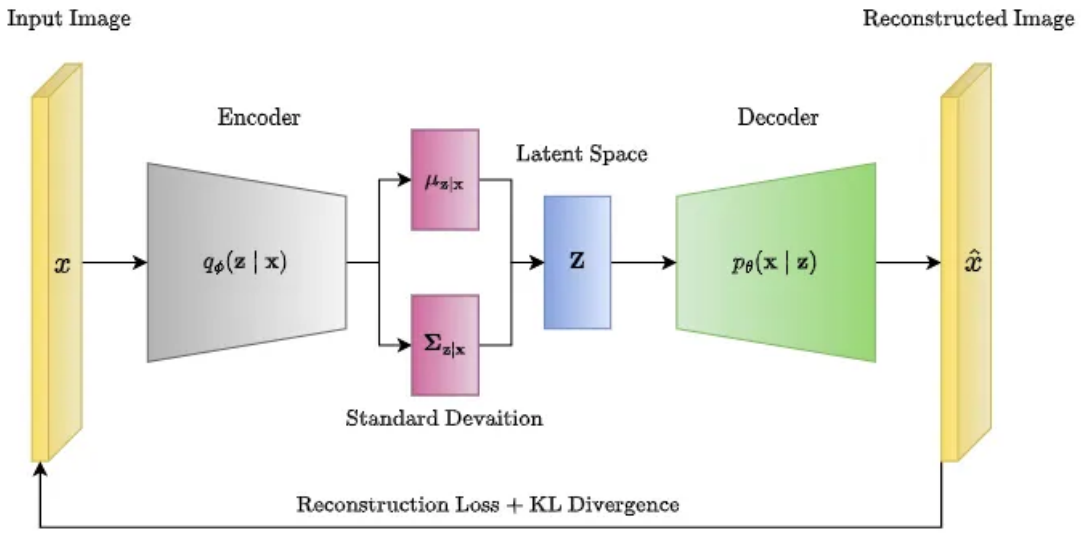

Figure 3.2: Structure of a Variational Auto-encoder, picture from [10]

The encoder receives as input a tensor representing an image extracted from a dataset, and it maps it into a distribution within the latent space, parametrized by a mean vector μ and a log-variance vector $\log \sigma^2$. The network predicts the logarithm of the variance instead of the standard deviation, so that its output

is unconstrained in sign while σ remains positive.

The decoder, instead, maps from the latent space back to the input space.

The VAE's parameters are optimized via gradient descent, however the sampling of the latent variable z introduces stochasticity, making the gradient of the loss with respect to the model parameters non-differentiable. A technique known as **Reparameterization Trick** is employed to address this problem, which expresses the sampling operation in a way that separates the deterministic and stochastic parts, enabling gradient flow through the deterministic part. So instead of directly sampling z as $z \sim N(\mu, \sigma^2)$, which is non-differentiable with respect to μ and σ, it sample an auxiliary variable ϵ from a standard normal distribution $N(0, I)$, and then it expresses z as $z = \mu + \sigma \cdot \epsilon$, which is now a differentiable function of μ and σ.

3.2.2 Components of VAE

The two primary components of the VAE are:

Encoder

The encoders architecture consists of four 2D convolution layers, one flatten layer and one dense layer. The input received by this module, which is a 1-channel 28×28 image, passes through the convolutional layers – using 32, 64, 64 and 64 filters respectively – to extract features, then the feature maps is flattened and its dimensionality is reduced using a dense layer; finally the encoder outputs the mean and the log-variance of the latent space distribution and samples the latent vector z.

Layer	Output shape	Parameters
Input	$28 \times 28 \times 1$	–
Convolution	$28 \times 28 \times 32$	320
Convolution	$14 \times 14 \times 64$	18,496
Convolution	$14 \times 14 \times 64$	36,928
Convolution	$14 \times 14 \times 64$	36,928
Flatten	12544	–
Dense	32	401,440
Dense	20	660
Dense	20	660
Sampling	20	–
Total		**495,432**

Table 3.3: Structure of the VAE encoder, shown for a latent dimension of 20. The two dense heads produce the mean and the log-variance of the latent distribution, and the sampling layer applies the reparameterization trick.

Decoder

The decoders architecture consists of a dense layer, a reshape layer and two 2D transposed convolution layers. The input of the module is the latent vector that, through the dense and reshape layers, it is transformed back into a tensor, matching the shape of the last convolution layer of the encoder; finally the convolution layers generate the reconstructed image with the same number of channels as the original input image.

Layer	Output shape	Parameters
Input	20	–
Dense	12544	263,424
Reshape	$14 \times 14 \times 64$	–
Transposed convolution	$28 \times 28 \times 32$	18,464
Transposed convolution	$28 \times 28 \times 1$	289
Total		**282,177**

Table 3.4: Structure of the VAE decoder, shown for a latent dimension of 20. With a latent dimension of 30 the total becomes 407,617 parameters.

3.2.3 Training Process

The optimization of VAE is achieved through the maximization of the **Evidence Lower Bound** (ELBO), consisting of two terms: the **Reconstruction Loss** and the **Regularization Term** (KullbackLeibler Divergence).
Given the input data x, the latent variable z, the encoder network $q_\phi(z|x)$, the decoder network $p_\theta(x|z)$, the prior distribution over the latent variables $p(z)$, the reconstruction loss $\mathbb{E}_{q_\phi(z|x)}[log\ p_\theta(x|z)]$, and the KL divergence $KL(q_\phi(z|x)\ ||\ p(z))$, the loss is computed as:

$$L(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[log\ p_\theta(x|z)] - KL(q_\phi(z|x)\ ||\ p(z)) \tag{3.2}$$

The VAE is trained with a custom loop, consisting of three main steps: **forward pass**, **loss calculation** and **backward pass**. During the forward pass, the input data is passed through the encoder to get the mean and log variance of the latent distribution, then the latent vector z is sampled using the reparamentrization trick, and finally z is passed through the decoder to get the reconstructed data. The loss calculation step computes the reconstruction loss between the input data and the reconstructed data, the KL divergence, and then sums them up to compute the total loss. Finally, in the backward pass the total loss is backpropagated to update the parameters of the encoder and decoder.

The model is trained for up to 100 epochs using a batch size of 100 and an Adam optimizer [6] with learning rate of 0.001, with early stopping monitoring the validation loss.

KL warm-up. Training the model as described above turned out to be unreliable: repeating it with different random seeds produced generators of very different quality, and the weaker runs showed **partial posterior collapse**, a known failure mode in which the encoder stops using part of the latent space and the decoder learns to ignore it. To counter it we scale the KL term by a coefficient that grows linearly from 0 to 1 over the first ten epochs. Early in training the latent code can become informative without being pulled back towards the prior, and the regularization reaches full strength only afterwards. The evaluation always uses a coefficient of 1, so validation losses remain comparable across epochs and across runs.
The effect is substantial. Without warm-up the average KL divergence at convergence was between 4 and 7 nats and the quality of the resulting prior varied by a factor of three across seeds; with warm-up it is between 16 and 23 nats and the spread across seeds falls to about 20%. The procedure used to choose

between runs is described in `docs/model_selection.md` in the repository.

To test the goodness of the model in the context of compress sensing, two versions of the VAE have been generated: one with the dimension of the latent vector equal to 20, and the other equal to 30. After using both of them, its possible to conclude that they lead to good results, as shown in the Results section.

3.2.4 The architecture of the reference paper

The convolutional networks described above are our own design. The experiments of Bora et al. [1] on MNIST instead use a **fully connected** auto-encoder, with a recognition network of shape $784 - 500 - 500 - k$ and a generator of shape $k - 500 - 500 - 784$, a latent dimension of 20, and the same optimizer, batch size and learning rate we use. Since our generator differs from theirs, a difference between our results and the numbers they publish could be attributed either to the method or to the model.
To separate the two, we also trained their architecture, and we report it as a separate prior in the Results section. The paper specifies the layer widths but not the activation function; we use softplus, following the original variational auto-encoder [7] the architecture is taken from.

3.3 Model results

The following graph shows the output mean vector produced by the encoder with the test set provided by the MNIST dataset. For the sake of simplicity, this result is obtained with a latent dimension equal to 2.

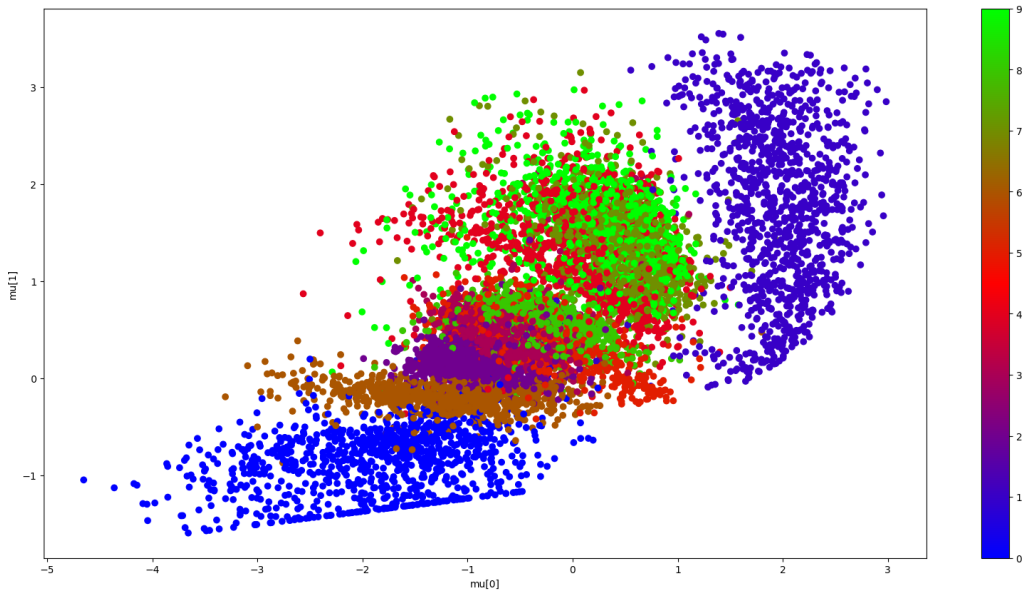

Figure 3.3: Plotted output of the encoder, each color represents a specific digit

By observing the data in the graph, we can predict the behaviour of the decoder. Infact, if we assume it receives as input the vector $[-2, -1]$, we expect the output to depict with good precision the number 0, as represented by the blue color in the lateral colorbar. The image reconstruction is shown in the image below.

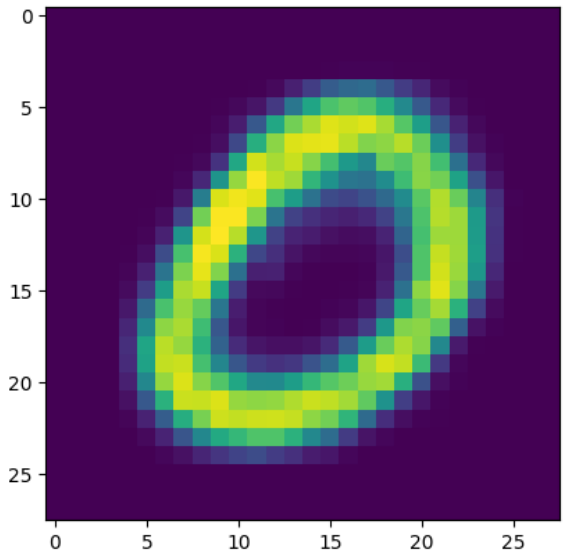

Figure 3.4: Output of the decoder for the input vector $[-2, -1]$

4 Algorithm Explanation

Generative models offer a powerful approach for compressed sensing by leveraging the structure of the data distribution.
Independently from the above mentioned models, a generative model is defined by a deterministic function $G : \mathbb{R}^k \to \mathbb{R}^n$ and a distribution P_Z over $z \in \mathbb{R}^k$. The model maps a low-dimensional latent vector z (decided by us as the latent dimension) to the high-dimensional sample space (the final generated image). The compressed sensing task involves finding a vector z in the latent space such that the corresponding generated sample $G(z)$ matches the observed measurements y.
The measurement matrix A is drawn at random, with entries sampled independently from a Gaussian distribution $N(0, 1/m)$. With this scaling the operator preserves norms in expectation, since $\mathbb{E}\|Ax\|^2 = \|x\|^2$ for every x, and the magnitude of the measurement error remains comparable across different values of m.
The objective function to be minimized is:

$$\text{loss}(z) = \|AG(z) - y\|^2. \tag{4.1}$$

The objective actually used in [1] adds a regularization term to this expression:

$$\text{loss}(z) = \|AG(z) - y\|^2 + \lambda\|z\|^2. \tag{4.2}$$

Since both the VAE and the DCGAN impose an isotropic Gaussian prior on z, the quantity $\|z\|^2$ is proportional to the negative log-likelihood of the latent vector under that prior, so the term keeps the search inside the region the generator was trained to cover. The authors report $\lambda = 0.1$ as the best value on MNIST and we adopt it, applying it to every generative prior for consistency. Results obtained with $\lambda = 0$ are reported alongside, as the reference paper does.
Optimization is performed using gradient descent on z with the Adam algorithm [6] for optimizing descent. If G is differentiable, gradients of the loss with respect to z can be computed using back-propagation. The process involves iteratively updating z to minimize the measurement error. Upon convergence to $\hat{z}$, the reconstruction of x^* is given by $G(\hat{z})$.

The algorithm can be summarized as follows:

1. Initialize z by sampling from the distribution P_Z, which for both models is the standard normal $N(0, I)$ used during training.

2. Compute the loss: $\text{loss}(z) = \|AG(z) - y\|^2$.

3. Use gradient descent to update z to minimize the loss.

4. Repeat until convergence.

5. The reconstructed signal is $\hat{x} = G(\hat{z})$.

This method leverages the data prior encoded in the generative model to achieve high-quality reconstructions with fewer measurements compared to traditional compressed sensing methods.

A theoretical justification is given in [1]. If G is L-Lipschitz, a number of random Gaussian measurements of the order of $k \log L$ is sufficient to guarantee that the reconstruction error is close to the smallest error achievable by any vector in the range of G. The number of measurements required therefore scales with the latent dimension k and not with the ambient dimension n, which is the reason why so few measurements are enough.
In order to improve convergence we performed a thousand gradient steps for each of ten random restarts, and we kept the attempt that presented the smallest measurement error $\|AG(z) - y\|$ at the end of the approximation. Note that the regularization term is excluded from this comparison: ranking the restarts on the full objective would reward a small $\|z\|$ rather than a faithful reconstruction. The measurement error is also the only criterion available in practice, since it can be computed without knowing x^*. These steps are not training epochs: the weights of G are frozen and the only variable that gets updated is the latent vector z.

5 The MNIST Dataset

The MNIST (Modified National Institute of Standards and Technology) dataset [8] is a large collection of handwritten digits commonly used for training various image processing systems. It serves as a benchmark for evaluating the performance of machine learning algorithms. The dataset contains 60,000 training images and 10,000 test images, each of which is a gray scale image of size 28×28 pixels.

Each image in the MNIST dataset is labeled with the digit it represents, ranging from 0 to 9. The dataset is particularly valuable because it provides a diverse set of handwriting styles, allowing for the development of robust digit recognition systems.

- **Size:** The training set consists of 60,000 images, and the test set consists of 10,000 images.
- **Resolution:** Each image is 28×28 pixels, making a total of 784 features.
- **Classes:** There are 10 classes, corresponding to the digits 0 through 9.
- **Grayscale:** Images are in grayscale, with pixel values ranging from 0 (black) to 255 (white).

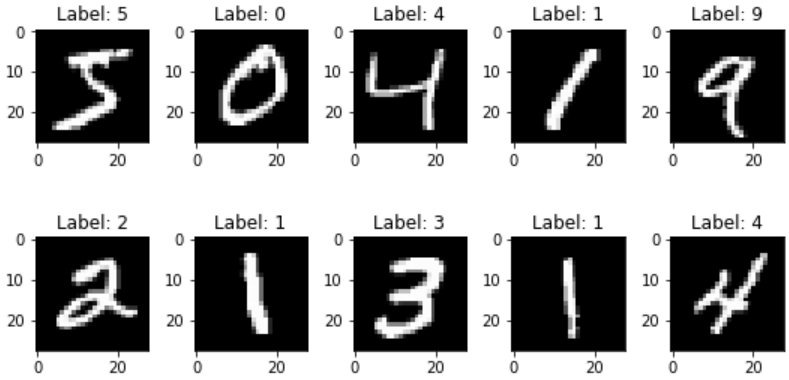

Figure 5.1: Sample of images from the MNIST dataset

6 Results

We compare five generative priors against two LASSO baselines. Four of the priors are the models described in the previous section, a VAE and a DCGAN each trained with a latent vector of dimension 20 and 30. The fifth is the fully connected architecture of Bora et al. [1], included so that the comparison with the numbers they publish is direct rather than indirect.

6.1 Experimental Protocol

Every method is evaluated on the same ten test images, one for each digit class, so that the comparison is not driven by a single lucky or unlucky example. At each measurement budget M all the methods receive the same measurement matrix A and the same noise vector, and the noise is scaled so that its expected norm remains equal to 0.1 whatever the value of M.
The reconstruction quality is measured as the squared distance between the reconstruction and the original image, divided by the number of pixels:

$$\text{error per pixel} = \frac{\|\hat{x} - x^*\|_2^2}{n}, \qquad n = 784. \tag{6.1}$$

This is not the same quantity as the measurement error $\|AG(\hat{z}) - y\|$ that the recovery algorithm minimizes. The latter decreases as M decreases, simply because fewer constraints have to be satisfied, so it describes how well the optimization converged rather than how good the reconstruction is. For this reason it is used only as a diagnostic and to rank the random restarts, while every number reported below is computed against the ground truth image.
All the reconstructions obtained with a generative prior use a thousand gradient steps, ten random restarts and the regularization weight $\lambda = 0.1$ recommended in [1]. Section 6.6 reports what happens without the regularization term.

6.2 The LASSO Baselines

LASSO (Least Absolute Shrinkage and Selection Operator) assumes sparsity in some basis Ψ, writes $x = \Psi\theta$, and solves

$$\hat{\theta} = \arg\min_{\theta} \|y - A\Psi\theta\|_2^2 + \alpha\|\theta\|_1, \qquad \hat{x} = \Psi\hat{\theta}. \tag{6.2}$$

We report two choices of Ψ. The first is the **pixel basis**, $\Psi = I$, which is the

baseline used in [1] for MNIST: the authors observe that the digits are already sparse in pixel space, since most of the image is background. The second is the **orthonormal DCT basis**, which the same authors use for natural images and which is the usual choice for image compression. The two behave very differently on this dataset and reporting only one of them would misrepresent how strong the classical approach is.
A baseline is only informative if it is given its best configuration, so the shrinkage α was swept over six orders of magnitude for each basis, and the value minimizing the mean error across budgets was kept. Both bases select $\alpha = 10^{-5}$. The reconstructions are clipped to $[0, 1]$, the range MNIST pixels live in, which improves the baseline. Note that the shrinkage quoted in [1] is not directly transferable, because the implementation we use scales the objective by $1/(2M)$. From our experiments, LASSO in either basis is inadequate until a substantial number of measurements is available, and neither produces a coherent digit below 200 measurements. On the other hand LASSO keeps improving as measurements are added, and in the pixel basis it becomes essentially exact once the budget approaches the dimension of the signal.

6.3 Comparison of the Methods

Table 6.1 and figure 6.1 report the error per pixel of every method as the number of measurements grows.

M	LASSO (pixel)	LASSO (DCT)	VAE (paper)	VAE k=20	VAE k=30	DCGAN k=20	DCGAN k=30
10	0.1309	0.1539	**0.0609**	0.0635	0.0766	0.1021	0.0833
25	0.1255	0.1049	0.0291	0.0372	**0.0225**	0.0619	0.0465
50	0.1172	0.0850	**0.0110**	0.0131	0.0177	0.0430	0.0347
75	0.1125	0.0707	**0.0079**	0.0111	0.0094	0.0248	0.0319
100	0.1048	0.0560	**0.0076**	0.0106	0.0080	0.0137	0.0291
200	0.0829	0.0366	**0.0067**	0.0097	0.0077	0.0106	0.0235
300	0.0406	0.0206	**0.0066**	0.0096	0.0074	0.0105	0.0247
400	0.0108	0.0117	**0.0066**	0.0096	0.0074	0.0098	0.0223
500	**0.0002**	0.0065	0.0066	0.0096	0.0074	0.0094	0.0238
750	**0.0000**	0.0015	0.0064	0.0094	0.0072	0.0095	0.0225

Table 6.1: Error per pixel, averaged over ten test digits, one per class. The best method at each budget is in bold. VAE (paper) is the fully connected architecture of [1].

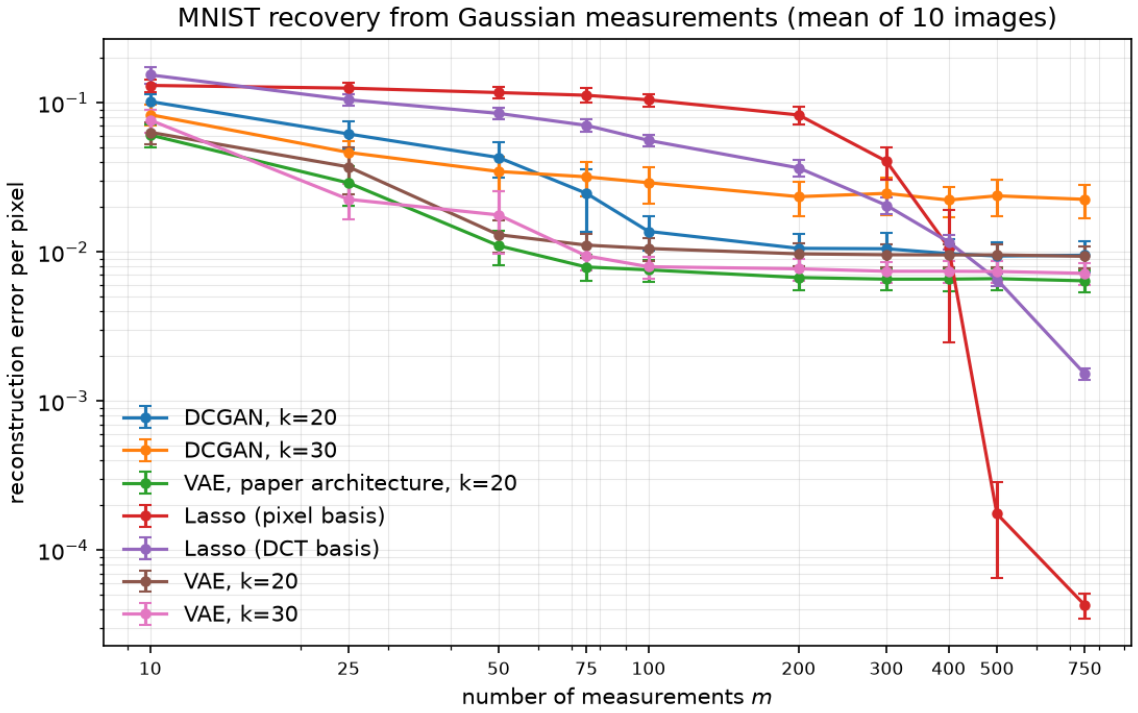

Figure 6.1: Error per pixel as a function of the number of measurements M. Bars are standard errors over the ten test images. Both axes are logarithmic.

Three regimes can be distinguished.

Few measurements. Up to about two hundred measurements every generative prior is more accurate than either baseline, and the best of them by a wide margin. With 25 measurements the best model has an error of 0.0225 against 0.1255 for LASSO in the pixel basis, a factor 5.6, and 0.1049 in the DCT basis, a factor 4.7. At that budget neither LASSO reconstruction is recognizable as a digit. This is the regime that motivates compressed sensing, and it is where the choice of the prior matters most.

Sample efficiency. Taking the baseline of the reference paper, LASSO needs 400 measurements to reach an error of 0.0108. The architecture of that paper reaches the same level with 75 measurements, a saving of a factor **5.3**, and so does our convolutional VAE with latent dimension 30. The convolutional VAE with latent dimension 20 needs 100 measurements, a factor 4, and the DCGAN with latent dimension 20 needs 200, a factor 2. Bora et al. report a saving between 5 and 10 times, and our reproduction of their architecture falls at the

lower end of that interval.

Many measurements. From 500 measurements onwards LASSO in the pixel basis becomes the most accurate method by a large margin, reaching an error below 10^{-4} with 750 measurements against 0.0064 for the best generative model. Once the number of measurements approaches the 784 dimensions of the signal, an assumption of sparsity in pixel space is enough to recover an MNIST digit almost exactly, while the generative prior remains stuck. The reversal itself is expected and is described in [1], where the authors observe that LASSO eventually surpasses the generative approach and that this requires more than 500 measurements. We observe it from 500 onwards, and in the DCT basis, which is the weaker baseline in this regime, from the same budget.

The reason for the reversal is that the reconstruction is constrained to lie in the range of the generator. The distance between a real digit and that range does not depend on the number of measurements, so once enough measurements are available to identify the closest point in the range, additional measurements bring no benefit. Averaging the budgets from 300 upwards, this residual error settles at 0.0065 for the paper architecture, 0.0074 for the VAE with latent dimension 30, 0.0095 for the VAE with latent dimension 20, 0.0098 for the DCGAN with latent dimension 20 and 0.0233 for the DCGAN with latent dimension 30. These five values are properties of the generators and not of the recovery algorithm.

6.4 Reconstruction Examples

Figure 6.2 shows the same digit reconstructed by every method at every budget.

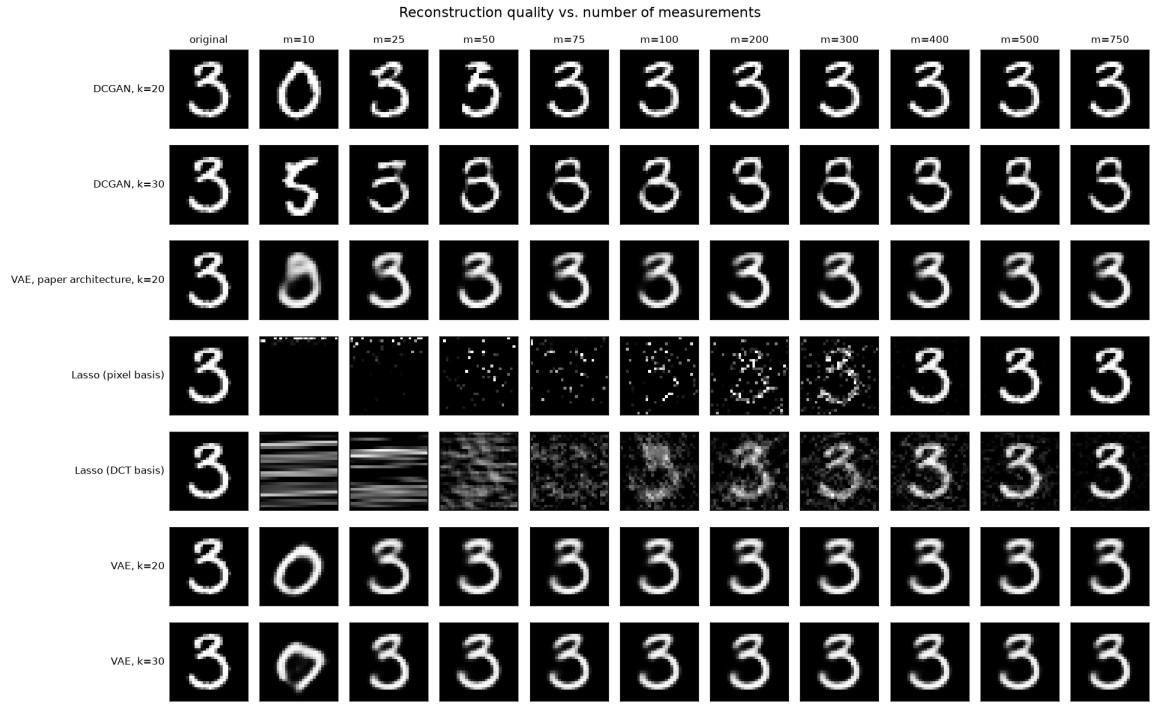

Figure 6.2: One test digit reconstructed by each method as the number of measurements increases

The visual comparison confirms the numbers. Both baselines return noise until about 200 measurements, while the generative priors produce a plausible digit almost immediately. The DCGAN gives visibly sharper strokes than the VAE, which is consistent with the fact that an adversarial generator is trained to produce samples a discriminator accepts, rather than an average of the plausible reconstructions as the VAE does. Sharpness and accuracy are not the same thing, however, and as the table shows the sharper model is the less accurate one.

6.5 Architecture and Latent Dimension

Because every method sees the same images and the same measurement matrices, the comparisons below are paired, and we test them with a paired t-test over the ten images at each budget.

The architecture of the paper outperforms ours. With the same latent dimension of 20, the fully connected network of [1] is more accurate than our convolutional VAE at every budget, and the difference is significant from 75 measurements upwards, with p-values at or below 0.004 and an advantage on 9

of the 10 test images. Below 75 measurements the difference is not significant. This is a result we did not expect, since the convolutional architecture is the more sophisticated of the two, and it suggests that what matters for this task is how faithfully the generator covers the data manifold rather than how well suited its architecture is to images.

A larger latent space helps, but only past the low measurement regime. For the convolutional VAE, the version with $k = 30$ is more accurate than the one with $k = 20$ from 75 measurements upwards, with a p-value of 0.028 at 75 and at or below 0.005 above it, and an advantage on 8 of the 10 images at 75 and on 9 of 10 at every larger budget. At 10, 25 and 50 measurements the difference is not significant and its sign is not even stable. This matches the intuition that a larger latent space represents digits more faithfully, which lowers the error floor, but offers no advantage while the measurements are too few to determine the extra coordinates.

The VAE beats the DCGAN, but not uniformly. At 10 and 50 measurements the convolutional VAE is significantly more accurate than the DCGAN of the same latent dimension, with p-values of 0.011 and 0.024. At the intervening budgets of 25, 75 and 100 the same comparison does not reach significance, so the advantage in the scarce regime is real but not established at every budget. From 200 measurements onwards the two are indistinguishable, both having settled on their floors.

6.6 The Latent Regularizer

Figure 6.3 compares the results obtained with $\lambda = 0.1$ and with the term removed.

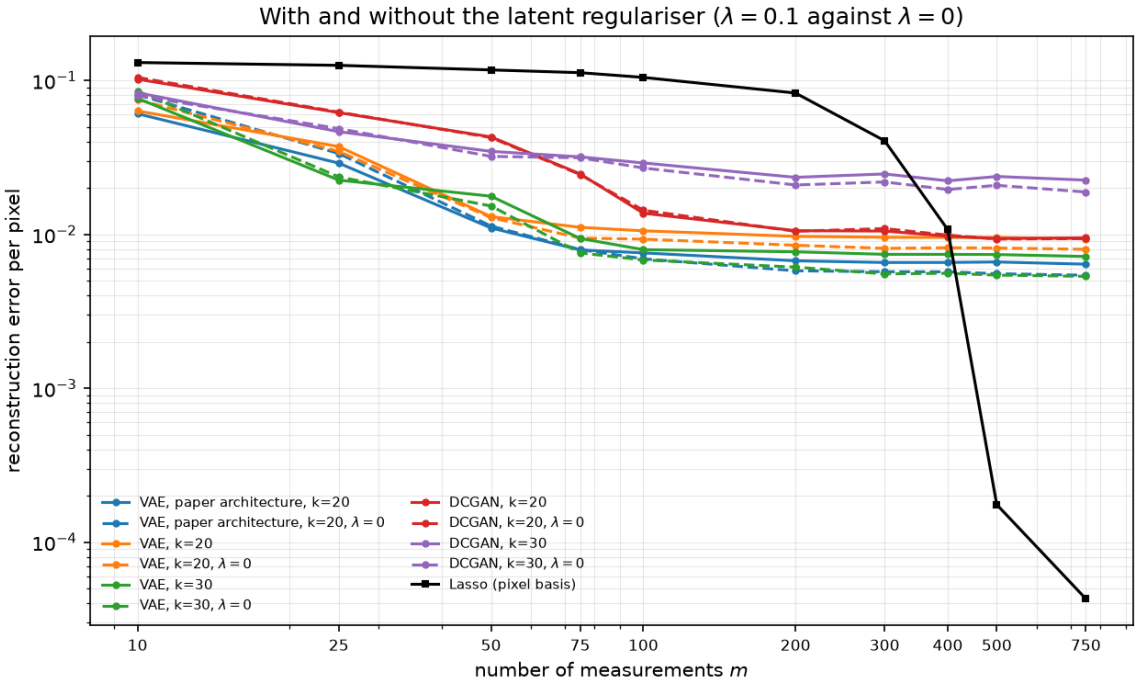

Figure 6.3: Recovery with and without the latent regularization term

The term behaves as its motivation suggests. It helps when the measurements are few, where the data alone does not determine the latent vector and the prior supplies the missing information: with 10 measurements the error of the paper architecture falls from 0.0808 to 0.0609, an improvement of 25%. It is mildly harmful once measurements are plentiful, where the same pull towards the prior keeps the solution from reaching the closest point in the range of the generator: with 750 measurements the error rises from 0.0054 to 0.0064.
A sweep over $\lambda \in \{0, 0.01, 0.1, 1\}$ makes the mechanism explicit. The norm of the recovered latent vector falls monotonically with λ, from 7.7 to 2.5 for the paper architecture, so the term does constrain the search as intended. More interestingly, the value of λ that minimizes the error is not fixed: it is 1 at 10 and 25 measurements, drops to 0.01 around 50 to 100, and is 0 for every budget of 200 and above. The single value recommended in [1] is therefore a compromise across regimes rather than an optimum at any one of them, which is consistent with the interpretation of the term as a substitute for missing measurements.

6.7 Computational Cost

The methods are not comparable in cost. Reconstructing the ten test images at one budget, with ten restarts and a thousand gradient steps, takes on average about one second with LASSO, 6.8 seconds with the fully connected decoder of the paper, about 15 seconds with our convolutional decoders and about 496 seconds with a DCGAN generator, a factor 73 between the cheapest and the most expensive generative prior.

The parameter counts do not explain that ratio: the DCGAN generator has 2.9 million parameters against 0.65 million for the fully connected decoder, a factor of only 4.5, and our convolutional decoder has fewer parameters than the fully connected one yet is more than twice as slow. What the cost tracks is the amount of arithmetic per forward pass. The fully connected decoder is three matrix products, whereas the DCGAN generator applies transposed convolutions with 256 and then 512 channels at close to full resolution, so each of its ten thousand forward and backward passes moves orders of magnitude more data.

This cost is incurred at every reconstruction and not only once during training, because recovery is itself an optimization problem. It is worth stressing that the most expensive prior is also the least accurate one at most budgets, so on this dataset there is no trade-off to arbitrate between the two. The sweep reported in this section required about three hours of computation, and the unregularized sweep of section 6.6 as much again.

6.8 Reproducibility of the Training

While preparing these results we found that training the same VAE with the same script and different random seeds produced generators of very different quality. Measured as the best reconstruction reachable inside the range of the generator, the results ranged from 0.0103 to 0.0358 for $k = 30$, a factor of 3.5, which is comparable to the entire spread between the five priors we set out to compare.

The cause is partial posterior collapse: in the weaker runs the decoder relies on a few latent directions and ignores the rest. Measuring how much each latent coordinate moves the generated image, the ratio between the average sensitivity and the largest one is 0.31 for the best model and 0.05 for the worst, and that ratio orders the models exactly as the reconstruction error does.

Adding a KL warm-up, as described in section 3.2.3, removes the problem. The spread across seeds falls from a factor 3.5 to about 20%, the KL divergence at convergence rises from between 4 and 7 nats to between 16 and 23, and the

resulting models are better than any obtained without it. The checkpoints used in this section were selected on validation loss alone, following a procedure fixed before the runs were carried out and recorded in the repository.
We report this because it affects how the comparisons above should be read: a difference between two priors is only meaningful when it is larger than the variability of the training procedure that produced them, which is why every comparison in section 6.5 is supported by a paired test rather than by a single pair of runs.

7 Conclusions

This project assessed the performance of compressed sensing with generative priors on the MNIST dataset, comparing a VAE and a DCGAN of our own design, the fully connected VAE of the reference paper, and LASSO in two sparsifying bases as the classical baseline.

Key Findings:

1. **The result of the reference paper is reproduced:**
    - With the architecture of [1], 75 measurements are enough to reach the accuracy their LASSO baseline requires 400 measurements to obtain, a saving of a factor 5.3. The authors report a saving between 5 and 10, so our reproduction falls at the lower end of the interval they publish.
    - From 500 measurements onwards LASSO becomes more accurate than every generative prior, which is the reversal the paper describes and places past the same threshold. In the pixel basis the reversal is dramatic: with 750 measurements LASSO recovers the digits almost exactly while the generative priors remain on their floors.
2. **The advantage has a ceiling, and the ceiling is the generator:**
    - The accuracy of the generative priors stops improving after roughly 200 measurements, because the reconstruction is constrained to the range of the generator and the distance between a real digit and that range does not depend on the number of measurements.
    - That floor ranges from 0.0065 for the best model to 0.0233 for the worst, a factor 3.6 between priors, so the choice of generator matters far more than any refinement of the recovery algorithm in this regime.
3. **The simpler architecture wins:**
    - The fully connected network of the reference paper is more accurate than our convolutional VAE at every budget, significantly so from 75 measurements upwards, and it is more than twice as fast to invert.
    - The DCGAN is the least accurate prior at almost every budget and by far the most expensive, about 73 times the cost of the fully

connected decoder per reconstruction. On this dataset there is no trade-off to arbitrate: the cheap model is also the good one.

4. **The training procedure is part of the result:**
   - Repeating the same VAE training with different random seeds produced generators whose quality varied by a factor of 3.5, comparable to the entire spread between the priors we set out to compare. The cause was partial posterior collapse.
   - Adding a KL warm-up removed the instability and improved every model. Any comparison between priors is only meaningful once it is larger than this variability, which is why the conclusions above are supported by paired tests rather than by single runs.

Future Directions:

- **Reducing the representation error:** the ceiling described above is a property of the generator, so the way to improve the method in the high measurement regime is a more expressive generative model rather than a better recovery algorithm.
- **Understanding why the fully connected model wins:** our convolutional decoder has fewer parameters devoted to the output layer and a stronger spatial inductive bias, and identifying which of the two explains the gap would say something useful about what makes a good prior for this task.
- **A budget dependent regularizer:** our sweep shows that the value of λ that minimizes the error decreases as measurements are added, from 1 at the smallest budgets to 0 at the largest, so a schedule rather than a constant is the natural refinement.
- **Optimizing measurement matrices:** designing or learning matrices adapted to a specific dataset, keeping in mind that the theoretical guarantee relies on the randomness of A and would be traded away.
- **Applications beyond MNIST:** MNIST digits form a simple and low dimensional manifold. Applying the method to more complex datasets would test whether the advantage in the low measurement regime survives.

In conclusion, combining a generative prior with compressed sensing is worthwhile exactly when measurements are the scarce resource, which is the situation

compressed sensing was designed for. On MNIST that regime ends well before the signal dimension is reached, and past it a classical sparsity assumption is not merely competitive but far better. The gain is bounded by how well the generator represents the data, and it is paid for with a reconstruction that is orders of magnitude more expensive than a convex solve. Both facts have to be weighed in applications such as medical imaging, where a single measurement can be slow, costly or harmful to the patient, and where the reconstruction happens once.

References

[1] Ashish Bora, Ajil Jalal, Eric Price, and Alexandros G. Dimakis. Compressed sensing using generative models. In *Proceedings of the 34th International Conference on Machine Learning*, pages 537–546, 2017. arXiv:1703.03208.

[2] Emmanuel J. Candès, Justin Romberg, and Terence Tao. Robust uncertainty principles: Exact signal reconstruction from highly incomplete frequency information. *IEEE Transactions on Information Theory*, 52(2):489–509, 2006.

[3] David L. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4):1289–1306, 2006.

[4] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems 27*, pages 2672–2680, 2014.

[5] KDnuggets. Generative adversarial networks: A hot topic in machine learning. `https://www.kdnuggets.com/2017/01/generative-adversarial-networks-hot-topic-machine-learning.html`, 2017.

[6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. arXiv:1412.6980.

[7] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014. arXiv:1312.6114.

[8] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[9] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In *International Conference on Learning Representations*, 2016. arXiv:1511.06434.

[10] Rushikesh Shende. Autoencoders, variational autoencoders (vae) and beta-vae. https://medium.com/@rushikesh.shende/autoencoders-variational-autoencoders-vae-and-beta-vae-ceba9998773d, 2023.

[11] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1):267–288, 1996.